

Inference Engineering

Assignment

1. Why is Inference Memory-Bandwidth Bound Important?

When a model is trained, it learns a large number of parameters or weights. During inference, we are not training the model anymore. We are using those already-learned parameters to generate an output.

For example, when I send a prompt to an LLM, the model has to process that prompt and generate the answer. In an autoregressive LLM, the answer is usually generated one token at a time. For every new token, the model performs many operations using its weights.

The important part here is that the model needs to continuously access its weights from memory while doing these computations. Modern GPUs are extremely fast at performing mathematical operations, but moving data from memory to the compute units can become the slower part.

This is where memory bandwidth becomes important.

Memory bandwidth basically tells us how quickly data can be transferred between the GPU's memory and the parts of the GPU that perform computation. It is usually measured in GB/s or TB/s.

For example, suppose a model has several gigabytes of weights. During inference, those weights need to be accessed repeatedly. If the GPU can perform calculations much faster than it can receive the required data, the computation units may sit idle while waiting for data.

In that situation, the inference workload is called **memory-bandwidth bound**.

A simple way I think about it is:

The GPU may know how to do the calculation very quickly, but it still has to wait for the data needed for that calculation to arrive.

This is especially important during LLM decoding because tokens are often generated sequentially. With a small number of requests being processed at the same time, there may not be enough parallel computation to fully utilize the GPU's compute capability.

This also explains why simply buying a GPU with more TFLOPS does not always mean that an LLM will generate tokens proportionally faster. If the main limitation is memory bandwidth, increasing compute capability does not solve the actual bottleneck.

There are two common types of bottlenecks:

- **Compute-bound:** The GPU spends most of its time performing mathematical operations.
- **Memory-bandwidth-bound:** The GPU spends more time waiting for data to be moved from memory.

So, understanding whether inference is compute-bound or memory-bandwidth-bound is important because it affects how we optimize the system.

For a company serving LLMs, this directly affects things such as:

- tokens generated per second,
- response latency,
- number of users that can be served,
- GPU utilization,
- and overall inference cost.

Therefore, memory bandwidth is not just a hardware specification. It can directly influence how efficiently an LLM can be served in the real world.

2. How Can We Calculate the Overall Memory Required to Store a Model? How Are LLMs Represented When Stored?

To calculate the memory required to store a model, I first need to understand what is actually being stored.

An LLM is mainly made up of a very large number of parameters, also called weights. These parameters are numerical values learned during training.

These numbers are not stored as normal decimal numbers such as the ones I would write in a notebook. They are stored digitally using a particular numerical format.

Some common formats are:

- FP32 – 32 bits per parameter
- FP16 – 16 bits per parameter
- BF16 – 16 bits per parameter
- INT8 – 8 bits per parameter
- INT4 – 4 bits per parameter

Since:

$$8 \text{ bits} = 1 \text{ byte}$$

the number of bytes required for one parameter depends on its numerical format.

For example:

$$\text{FP32} = \frac{32}{8} = 4 \text{ bytes}$$

and

$$\text{FP16} = \frac{16}{8} = 2 \text{ bytes}$$

Therefore, a simple estimate for the memory required to store the model weights is:

$$\text{Memory required} = \text{Number of parameters} \times \text{Bytes per parameter}$$

For example, consider a model with 7 billion parameters.

If it is stored using FP32:

$$7 \times 10^9 \times 4 = 28 \times 10^9 \text{ bytes}$$

which is approximately 28 GB.

If the same model is stored using FP16:

$$7 \times 10^9 \times 2 = 14 \times 10^9 \text{ bytes}$$

which is approximately 14 GB.

Similarly, using INT8 would require approximately 7 GB, while INT4 would require approximately 3.5 GB for the raw parameter data.

How is an LLM actually represented when stored?

This was important for me to understand because saying “a 7B model requires 14 GB” does not really explain what is inside those 14 GB.

An LLM does not exist as one huge list of numbers. Its parameters are organized into different **tensors**. A tensor can basically be thought of as a multidimensional array of numbers.

For example, a Transformer model contains tensors for different parts of the architecture. These can include:

- token embeddings,
- attention projection weights,
- query, key, and value weights,
- output projection weights,
- feed-forward or MLP weights,
- normalization parameters,
- and other model-specific parameters.

For example, attention may contain weight matrices commonly represented as:

$$W_Q, \quad W_K, \quad W_V, \quad W_O$$

where these matrices contain the numerical parameters used by the attention mechanism.

When the model is saved to disk, these tensors are stored as binary numerical data along with information describing the tensors, such as their names, shapes, and data types.

This is why model formats such as **safetensors**, **GGUF**, and other model formats contain much more than just a model name. They store the actual numerical data required to reconstruct the model.

It is also important to distinguish between **model storage** and **total inference memory**.

The equation:

$$\text{Parameters} \times \text{Bytes per parameter}$$

mainly gives us the memory required for the model's weights.

During actual inference, additional memory is required for things such as:

- KV cache,
- activations,
- temporary computation buffers,
- runtime overhead,
- and other framework-specific memory requirements.

So the actual GPU memory required to run an LLM can be larger than the size of the model file itself.

This is also connected to the previous question about memory bandwidth. During inference, the system does not just need enough memory to store the model. It also needs to move the required data between memory and the GPU's compute units efficiently.

3. Explain Each Layer of Inferencing and Its Importance to a Company

When I look at LLM inference from a company perspective, I find it easier to divide the whole system into three main layers:

1. Runtime
2. Infrastructure
3. Tooling

These three layers are connected, but they solve different problems.

3.1 Runtime

The runtime is basically the software responsible for actually running the model.

Once I have a trained model, I need something that can load its weights, execute the model's operations, manage the generation process, and communicate with the available hardware.

Examples of inference runtimes and serving systems include vLLM, TensorRT-LLM, llama.cpp, and other similar systems.

The runtime handles things such as:

- loading model weights,
- executing the model operations,
- managing token generation,
- managing the KV cache,
- batching requests,
- scheduling requests,
- and interacting with GPUs and other hardware.

The main question that the runtime answers is:

How can I execute this model as efficiently as possible?

This is extremely important for a company because a better runtime can improve throughput and latency without changing the actual model.

For example, if a company has thousands of users sending requests to an LLM, inefficient request scheduling or poor memory management can result in lower GPU utilization and higher costs.

3.2 Infrastructure

Infrastructure is the physical and system-level environment in which the inference runtime operates.

This includes things such as:

- GPUs or other accelerators,
- servers,
- CPU and system memory,
- networking,
- storage,
- load balancers,
- clusters,
- and autoscaling systems.

For example, if a company wants to serve a large LLM to millions of users, running one GPU on one machine is obviously not enough. The company may need many GPU servers working together.

The infrastructure layer is mainly concerned with questions such as:

What hardware and system architecture do I need to run this model reliably and at scale?

Infrastructure decisions have a direct impact on cost, reliability, scalability, and latency.

For example, choosing a GPU with enough VRAM to hold the model is only the beginning. The company also needs to consider networking between machines, power consumption, availability, scaling, failures, and how requests are distributed across the available hardware.

3.3 Tooling

Tooling is the layer that helps engineers understand, measure, debug, and optimize the inference system.

It is important to distinguish tooling from the model API.

For example, if I use an API provided by a company such as Google to access Gemini, that API is an **interface** through which my application communicates with the company's inference service. It is not itself the inference tooling.

A simplified flow would be:

My Application → API → Inference Service → Runtime → GPU → Model

Tooling exists around this system to help engineers understand what is happening.

Examples include tools for:

- benchmarking,
- profiling GPU usage,
- measuring latency,
- measuring throughput,
- monitoring memory usage,
- logging,
- tracing,
- detecting bottlenecks,
- and comparing different inference configurations.

The main question tooling answers is:

How do I measure, debug, and optimize the inference system?

This is important for a company because without proper measurement, it becomes difficult to know where money and performance are being lost.

For example, suppose an LLM application becomes slow. Tooling can help determine whether the problem is caused by GPU computation, memory bandwidth, the KV cache, networking, request scheduling, or some other part of the system.

3.4 Putting the Three Layers Together

The easiest way I understand these three layers is:

Layer	Main Question
Runtime	How do I execute the model efficiently?
Infrastructure	What system do I need to run it at scale?
Tooling	How do I measure, debug, and optimize it?

These layers are important because running an LLM in production is not simply a matter of downloading a model and putting it on a GPU.

A company has to think about the entire system.

The **runtime** determines how efficiently the model is executed. The **infrastructure** provides the hardware and systems needed to serve users reliably and at scale. The **tooling** helps engineers measure the system and find ways to improve it.

So, from a company perspective, all three layers ultimately affect the same things: **performance, scalability, reliability, and cost**.